Distribution Agreement

In presenting this thesis as a partial fulfillment of the requirements for a degree from Emory University, I hereby grant to Emory University and its agents the non-exclusive license to archive, make accessible, and display my thesis in whole or in part in all forms of media, now or hereafter now, including display on the World Wide Web. I understand that I may select some access restrictions as part of the online submission of this thesis. I retain all ownership rights to the copyright of the thesis. I also retain the right to use in future works (such as articles or books) all or part of this thesis.

Signature:

YiFei Guan April 02, 2026

Stochastic Rounding in Mixed-Precision Iterative Refinement

By

YiFei Guan

James G. Nagy, Ph.D.
Advisor

Department of Mathematics

James G. Nagy, Ph.D.
Advisor

Julianne Chung, Ph.D.
Committee Member

Adam Glynn, Ph.D.
Committee Member

2026

Stochastic Rounding in Mixed-Precision Iterative Refinement

By

YiFei Guan

James G. Nagy, Ph.D.
Advisor

An abstract of
a thesis submitted to the Faculty of Emory College of Arts and Sciences of
Emory University in partial fulfillment
of the requirements of the degree of
Bachelor of Science with Honors

Department of Mathematics

2026

Abstract

Stochastic Rounding in Mixed-Precision Iterative Refinement
By YiFei Guan

In this thesis, we study the use of stochastic rounding (SR) in mixed-precision iterative refinement (IR) algorithms for solving linear systems and least squares problems. Tikhonov regularization in block form is used to address ill-posed and noisy problems. In particular, SR is applied in the low-precision computation step of the mixed-precision IR algorithms, where it has been shown to yield smaller backward error bounds with high probability and is therefore less susceptible to stagnation. Numerical experiments on extremely ill-posed inverse heat problems and a large-scale PRblur Kronecker product problem are used to demonstrate its practical value. We first find that regularization is most beneficial for ill-posed and noisy problems, while otherwise increasing computational complexity with limited advantage. For the PRblur problem, we observe that mixed-precision IR using SR in the low-precision solve performs comparably to pure higher-precision computation, indicating its potential for real-world applications. These experiments also highlight current limitations, including the constraints of the Julia stochastic rounding package and the general lack of GPU support for SR. Future work may examine GPU-based implementations of SR and their impact on runtime and convergence.

Stochastic Rounding in Mixed-Precision Iterative Refinement

By

YiFei Guan

James G. Nagy, Ph.D.
Advisor

A thesis submitted to the Faculty of Emory College of Arts and Sciences
of Emory University in partial fulfillment
of the requirements of the degree of
Bachelor of Science with Honors

Department of Mathematics

2026

Acknowledgments

I would like to thank my advisor, Dr. Nagy, for his guidance, patience, and remarkable ability to tolerate my procrastination.

I am especially grateful to my parents for their generous financial support, unknowingly helping fund the Chipotle and McDonald's that kept me warm.

Lastly, I thank my dog, Jasper, for his constant company and for being the greatest distractions during the writing of this thesis.

Contents

1 Introduction **1**

1.1 Iterative Refinement 3

1.2 Floating Point Arithmetic 4

1.3 Numerical Examples 6

1.4 Outline of Thesis 8

2 Literature Review **9**

3 Rounding in Floating Point Arithmetic **13**

3.1 Round-to-Nearest and Stochastic Rounding 13

3.2 Stochastic Rounding Packages 15

4 Iterative Refinement **17**

4.1 IR Algorithms 17

4.1.1 Factorization 18

4.1.2 GMRES 19

4.2 Regularization 20

5 Numerical Experiments **22**

5.1 Ill-conditioned inverse heat matrix 23

5.2 PRblur Kronecker product 25

5.2.1 Kronecker product definition 26

5.2.2 Noise free condition 27
5.2.3 Noise added conditions 29
5.2.4 Image display 33

6 Concluding Remarks **35**

Bibliography **38**

List of Figures

5.1 Comparison of inverse heat matrix runs with LU and GMRES 24
5.2 PRblur noise-free single precision 27
5.3 PRblur noise free mixed-precision 28
5.4 PRblur small noise single precision 29
5.5 PRblur medium noise single precision 30
5.6 PRblur large noise single precision 30
5.7 PRblur small noise mixed-precision 31
5.8 PRblur medium noise mixed-precision 32
5.9 PRblur large noise mixed-precision 32
5.10 PRblur true figure 33
5.11 PRblur noise free and large noise restoration 34

Chapter 1

Introduction

In this thesis, we consider using mixed-precision iterative refinement algorithms to solve the linear equation

$$\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b} \tag{1.1}$$

and ill-conditioned problems can be dealt with Tikhonov regularization in block form,

$$\begin{bmatrix} \boldsymbol{I} & \boldsymbol{A} \\ \boldsymbol{A}^T & -\lambda^2\boldsymbol{I} \end{bmatrix} \begin{bmatrix} \boldsymbol{r} \\ \boldsymbol{x} \end{bmatrix} = \begin{bmatrix} \boldsymbol{b} \\ \boldsymbol{0} \end{bmatrix} \tag{1.2}$$

which is equivalent of working with the augmented system,

$$\begin{aligned} \boldsymbol{r} + \boldsymbol{A}\boldsymbol{x} &= \boldsymbol{b} \\ \boldsymbol{A}^T\boldsymbol{r} - \lambda^2\boldsymbol{x} &= \boldsymbol{0} \end{aligned} \tag{1.3}$$

Regularization can be implemented in many other instances apart from using the block form of Tikhonov regularization. Details will be introduced and explained in Chapter 4 with other types of regularization and how it can be used.

Further down in the thesis within algorithms, we also solve least square problems,

$$\min \|\boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}\|_2^2 \tag{1.4}$$

with respected Tikhonov regularization form [6],

$$\min \|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2 + \lambda^2 \|\boldsymbol{x}\|_2^2 = \min \left\| \begin{bmatrix} \boldsymbol{b} \\ \boldsymbol{0} \end{bmatrix} - \begin{bmatrix} \boldsymbol{A} \\ \lambda \boldsymbol{I} \end{bmatrix} \boldsymbol{x} \right\|_2^2 \tag{1.5}$$

where $\boldsymbol{b} \in \mathbb{R}^m$, $\boldsymbol{x} \in \mathbb{R}^n$, λ is a scalar, $\boldsymbol{I} \in \mathbb{R}^{n \times n}$ is the identity matrix, and $\boldsymbol{A} \in \mathbb{R}^{m \times n}$.

It is clear that if $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ and is non-singular, then equation (1.1) and equation (1.4) solve the same problem. Below we show that equation (1.2) and equation (1.5) also solve the same problem.

Looking at equation (1.5), we can solve the regularized least squares problem by the normal equations. Thus, we can rewrite (1.5) as

$$\begin{aligned} \begin{bmatrix} \boldsymbol{A}^T & \lambda \boldsymbol{I} \end{bmatrix} \begin{bmatrix} \boldsymbol{A} \\ \lambda \boldsymbol{I} \end{bmatrix} \boldsymbol{x} &= \begin{bmatrix} \boldsymbol{A}^T & \lambda \boldsymbol{I} \end{bmatrix} \begin{bmatrix} \boldsymbol{b} \\ \boldsymbol{0} \end{bmatrix} \\ \left(\boldsymbol{A}^T \boldsymbol{A} + \lambda^2 \boldsymbol{I} \right) \boldsymbol{x} &= \boldsymbol{A}^T \boldsymbol{b} \end{aligned} \tag{1.6}$$

At the same time, we can also solve equation (1.3) by substituting $\boldsymbol{r} = \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}$ into the other equation in the system, which gives

$$\begin{aligned} \boldsymbol{A}^T (\boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}) - \lambda^2 \boldsymbol{x} &= \boldsymbol{0} \\ \boldsymbol{A}^T \boldsymbol{b} &= \left(\boldsymbol{A}^T \boldsymbol{A} + \lambda^2 \boldsymbol{I} \right) \boldsymbol{x}. \end{aligned} \tag{1.7}$$

The results from equation (1.6) and equation (1.7) are the same, indicating that mathematically they solve the same problem but in different formats.

1.1 Iterative Refinement

To solve the linear equation or least squares problem proposed above, iterative refinement (IR) will be used in this thesis. IR is an algorithm for improving an approximate solution to a linear system by repeatedly computing the residual and using it to form a correction [14]. Starting from an initial guess, the method evaluates how far the current solution is from satisfying the system, solves for an update, and then adds it to the current approximation. By repeating this process, the solution is incrementally corrected and ideally moves closer to the true solution.

IR is useful because it can improve the accuracy of an approximate solution through successive corrections without recomputing the entire solution from scratch at each step. Although each iteration still requires solving a system of the form $\boldsymbol{Ad} = \boldsymbol{r}$, this step is typically performed using a more efficient or previously computed structure (such as a factorization or iterative method). As a result, IR reuses information from earlier computations and only computes a correction, making the process more efficient while still progressively improving the solution.

There are many different IR algorithms but their basic forms are similar. The one we reference to in this thesis is organized by Higham and Mary as *Algorithm 6.1* in their paper [11]. In solving the system, Algorithm 1 is the fundamental structure of all variations of the algorithms that will be presented in this paper.

Algorithm 1 Given a non-singular matrix $\boldsymbol{A} \in \mathbb{R}^{n\times n}$, $\boldsymbol{b} \in \mathbb{R}^n$, and an initial guess $\boldsymbol{x}_1$, this algorithm uses mixed-precision iterative refinement to solve $\boldsymbol{Ax} = \boldsymbol{b}$. Three precisions are used in the algorithm where $u_r \leq u \leq u_l$.

---

1 for $i = 1 : i_{\max}$ or until relative residual reaches tolerance
2 Compute $\boldsymbol{r}_i = \boldsymbol{b} - \boldsymbol{Ax}_i$ in precision u_r.
3 Solve $\boldsymbol{Ad}_i = \boldsymbol{r}_i$ at precision u_l.
4 Update $\boldsymbol{x}_{i+1} = \boldsymbol{x}_i + \boldsymbol{d}_i$ in precision u.
5 end

For my implementation in Chapter 5, the precision choice is fixed, where $u_r \approx$

10^{-16} (double precision or Float64), $u \approx 10^{-8}$ (single precision or Float32), and $u_l \approx 10^{-4}$ (half precision or Float16). In some examples, we will also compare how the algorithm performs in single-precision to indicate the advantage of mixed-precision calculation.

In our implementation with a fixed combination of precisions, we first choose an initial guess $\boldsymbol{x}_1$ in Float64. We then compute the residual of the initial guess $\boldsymbol{r}$ also in Float64. Next, compute the update $\boldsymbol{d}$ using different methods in Float16. Finally, we update the solution $\boldsymbol{x}$ in Float32. The general implementation in Algorithm [11] has less constraint on the use of precision, with the update $\boldsymbol{d}$ generally computed in a lower precision than the other two steps.

The key modification in this paper is the manipulation of the second step of the algorithm, the calculation of $\boldsymbol{d}$. Since this is the step that takes up most of the computation, working in a low precision supports efficiency. Different methods, from basic factorization, like LU or QR, to iterative methods, including GMRES, are introduced for this step. More detail will be presented in Chapter 4. Moreover, stochastic rounding has its advantages in low precision calculations, which can also be implemented at this step. We will briefly introduce stochastic rounding in the following subsection, and more details will be discussed in Chapter 3.

1.2 Floating Point Arithmetic

Floating-point arithmetic is a numerical representation system used in modern computation to approximate real numbers within a finite number of bits [8]. Instead of storing a number exactly, it is represented in a normalized scientific notation

$$x = \pm m \times \beta^e, \tag{1.8}$$

where β is the base (usually $\beta = 2$ in modern computers), m is the significance (or mantissa), and e is the exponent. This representation allows a wide range of magnitudes to be stored, but introduces rounding errors because only a finite set of numbers can be represented.

In practice, any real number x is mapped to the nearest representable floating-point number through a rounding function,

$$\mathrm{fl}(x) = x(1+\delta), \qquad |\delta| \leq \varepsilon, \tag{1.9}$$

where ε is the machine epsilon, representing the upper bound on relative rounding error. We discuss in more detail arithmetic with floating point numbers, or floating point operations (FLOPS) in Chapter 3.

While floating-point arithmetic enables efficient computation, it also introduces rounding errors that can accumulate, especially in ill-conditioned problems. Understanding this behavior is essential when designing stable numerical algorithms.

Round-to-nearest (RTN) is a deterministic rounding rule that maps a real number to the nearest representable floating-point number. If the number lies exactly halfway between two representable values, a tie-breaking rule is applied. This ensures consistency and minimizes the rounding error for each operation. As a result, the rounding error is bounded but deterministic.

A main consideration of this thesis is the use of **stochastic rounding** (SR), a rounding method that randomly rounds a real number to one of its nearest floating-point neighbors with probabilities proportional to their distances.

$$\mathrm{SR}(x) \begin{cases} P(SR(x) = \lfloor x \rfloor) = 1 - \dfrac{x - \lfloor x \rfloor}{\lceil x \rceil - \lfloor x \rfloor}, \\ P(SR(x) = \lceil x \rceil) = \dfrac{x - \lfloor x \rfloor}{\lceil x \rceil - \lfloor x \rfloor}. \end{cases} \tag{1.10}$$

where $\lfloor x \rfloor$ and $\lceil x \rceil$ denote the adjacent representable floating-point numbers in the working precision. If $\mathbb{F}$ is the set of representable floating-point numbers, we define the floor function as $\lfloor x \rfloor = \max\{y \in \mathbb{F} \mid y \leq x\}$, and the ceiling function as $\lceil x \rceil = \min\{y \in \mathbb{F} \mid y \geq x\}$.

From past papers, stochastic rounding (SR) yields backward error bounds that scale like $\sqrt{n}u$ (with high probability) rather than nu, where n is the problem size and u is the unit round-off, without requiring additional assumptions on the rounding error model [2] [3]. These bounds are typically derived for inner product computations, which are fundamental building blocks in linear algebra algorithms. Thus, in theory, SR is immune to stagnation during small solution updates, making it appealing for iterative algorithms [2] [3] [11]. Moreover, the rounding logic of SR works better with low precisions where errors grow rapidly, allowing it to mitigate error accumulation while preserving small updates [2] [3].

Based on Algorithm 1, the calculation of the update $\boldsymbol{d}$ is performed in low precision (Float16) for efficacy purpose, which is the perfect step for SR to step in. Therefore, throughout the numerical experiments in Chapter 5, we will be comparing results from round-to-nearest (RTN) and stochastic rounding (SR) at this stage. SR will be examined for its advantages in low-precision computations, including preserving small updates, mitigating stagnation, and offering new perspectives on mixed-precision iterative refinement. More on the logistics of SR, the packages used in this thesis, and additional considerations are discussed in detail in Chapter 3.

1.3 Numerical Examples

All numerical experiments in this thesis are run on the **NVIDIA H100** computing system at Emory. The connection is established through VS Code to the ADA server. Only the CPU will be used as the GPU is not compatible with the Julia SR package.

The two main numerical examples we will use to test the effectiveness of SR are inverse heat matrices and an image deblurring PRblur Kronecker product matrix. Details on constructions of problems will be explain in Chapter 5. In both cases, we tested regularized and un-regularized solutions. For the PRblur example, we also tested cases with additional noise added. As stated in the beginning of this thesis, we will be using the blocked form of Tikhonov regularization on ill-posed problems.

The reasons for using these examples are clear. SR is theoretically useful in preventing stagnation, which is more likely to occur when the problem is ill-posed. When dealing with extremely ill-posed problems, regularization is a natural choice to reduce the condition number. The blocked form of Tikhonov regularization also provides a special structure to the problem matrix, allowing the algorithm to operate more efficiently [9]. The inverse heat matrix serves as a test on a smaller problem and allows comparison of how SR performs in a generic setting. The PRblur example demonstrates SR's performance on larger problems. In addition, we defined the Kronecker product as an object in Julia by translating similar capabilities in the MATLAB IR-tools package [7], enabling the algorithm to efficiently work with Kronecker products implicitly.

Apart from comparing rounding methods, the algorithm will also be performed in single-precision to illustrate the advantages that mixed-precision brings to iterative refinement. Although this is not the primary focus of the thesis, the single-precision results serve as a baseline for comparison with the mixed-precision RTN and SR algorithms.

To compare the results, we mainly rely on how the relative error decreases. Graphs are plotted on a logarithmic scale to visualize, and the deblurred image will be displayed for the PRblur example. In addition, runtime analysis will be presented, though its implications are limited because the random number generator (RNG) in the SR package cannot run in parallel, significantly increasing the runtime. More will

be explained in Chapter 3.

In the numerical experiments, most codes and packages, like the Kronecker object definitions, are a direct translate of MATLAB codes from IRtools. More explanation will be presented in Chapter 5.

1.4 Outline of Thesis

This thesis is structured as follows:

In this introductory Chapter 1, we briefly outline the questions addressed in this thesis and the main algorithm used. Chapter 2 reviews key literature that provides the theoretical foundation for this work. Chapter 3 discusses IEEE standards for floating point numbers and compares round-to-nearest (RTN) to stochastic rounding (SR), including the implementation of the stochastic rounding package. Chapter 4 describes the iterative refinement approach for solving linear systems, with a focus on the structure and variations of different algorithms. Numerical examples are presented in Chapter 5, detailing the setup of various problem types and the application of Kronecker products. Finally, Chapter 6 offers concluding remarks and discusses the proposed methodologies of this thesis.

Chapter 2

Literature Review

In this chapter, we review the major literature that forms the theoretical foundation of this thesis. In total, six papers are discussed to provide background for the direction of this thesis. Additional papers are cited for their methodologies and technical results but are not formally discussed here. The literature here is organized by publication year to highlight the development of methodologies in this field.

Accurate and Efficient Floating-Point Summation (2003)

Demmel and Hida showed that naive summation can suffer severe roundoff errors as the number of summands increases. They proposed the use of wider-precision accumulators to significantly reduce roundoff error without resorting to expensive compensation techniques. Their analysis demonstrated that accurate summation can be achieved to within approximately 1.5 units in the last place under appropriate accumulation strategies. This work provided a foundational understanding of how rounding errors arise in inner products and residual computations, and it directly motivated mixed-precision IR methods that rely on higher-precision accumulation to maintain accuracy [5].

Kronecker Product Approximations for Image Restoration with Reflexive Boundary Conditions (2004)

Nagy et al. showed that the associated blurring matrices possess an exploitable Kronecker product structure. They proposed an efficient method to compute approximate Kronecker decompositions by reducing the problem to a small SVD form, therefore avoiding operations on prohibitively large systems. This approach enabled fast solution methods and regularization techniques, such as Tikhonov regularization, while significantly reducing computational costs. The paper highlighted how structural properties of ill-posed problems can be leveraged to improve numerical efficiency and stability, making such matrices well suited for studying convergence behavior and stagnation phenomena in mixed-precision IR algorithms [15].

Stochastic Rounding and Its Probabilistic Backward Error Analysis (2021)

Connolly et al. established a theoretical framework in which rounding errors are modeled as unbiased random variables rather than as deterministic truncation effects. The analysis showed that SR leads to unbiased error propagation and provides probabilistic guaranties on numerical accuracy. They explained why SR avoids systematic error accumulation and stagnation that commonly arise under RTN arithmetic in low precision. It has been shown that the rounding error bound can be improved by replacing nu by $\sqrt{nu}$ where n denotes the problem size and u denotes the unit roundoff. Here, the unit roundoff u is defined as the maximum relative error introduced by rounding a real number to the nearest floating-point number. These results offer a strong theoretical foundation for the use of SR in mixed-precision IR algorithms [2].

Stochastic Rounding: Implementation, Error Analysis, and Applications (2022)

Croci et al. investigated SR from both algorithmic and practical perspectives, focus-

ing on its implementation, error behavior, and applications in numerical algorithms. The paper showed that SR improves numerical stability in low-precision arithmetic by eliminating systematic bias in rounding errors, which in turn leads to improved convergence of iterative methods. Through both theoretical analysis and numerical experiments, they demonstrated that SR is particularly effective in preventing stagnation when updates fall below machine epsilon under deterministic rounding. The work also emphasized the compatibility of SR with mixed-precision computing, making it well suited for modern hardware architectures [3].

Effects of round-to-nearest and stochastic rounding in the numerical solution of the heat equation in low precision (2022)

Croci and Giles studied the impact of RTN and SR in low-precision numerical solutions of the heat equation using Runge–Kutta finite difference methods. Their results show that the RTN method leads to the accumulation of rounding errors and stagnation as the timestep decreases, with the global error scaling as $O(u\Delta t^{-1})$. In contrast, SR introduces zero-mean rounding errors that prevent systematic accumulation, leading to significantly improved error behavior. In particular, the rounding error grows more slowly, scaling as $O(u\Delta t^{-1/4})$ in one dimension and remaining essentially bounded in higher dimensions. These findings demonstrate that SR can improve numerical stability in low-precision computations and support its use in mixed-precision numerical algorithms [4].

Mixed Precision Low-Rank Approximations and Their Application to Block Low-Rank LU Factorization (2022)

Amestoy et al. proposed that singular vectors associated with small singular values can be stored and manipulated in lower precision without compromising the overall accuracy of the approximation. They provided a rigorous rounding-error analysis

showing that errors introduced by reduced precision are naturally damped by the spectral decay of the matrix. Using this principle, they developed a mixed-precision BLR LU factorization that preserves numerical stability while achieving up to three times reduction in memory usage and computational cost. This work demonstrates that precision can be adaptively matched to spectral content, offering a powerful approach for accelerating large-scale linear algebra while retaining accuracy [1].

Mixed Precision Algorithms in Numerical Linear Algebra (2022)

Higham and Mary presented a comprehensive framework for mixed-precision algorithms in numerical linear algebra, covering matrix factorizations, Krylov subspace methods, and iterative refinement. The paper analyzed how low-precision arithmetic can be combined with higher-precision corrections to achieve both efficiency and numerical stability. In particular, the authors showed that LU-based iterative refinement can attain high-precision accuracy even when the factorization is performed in low precision, provided that residuals are computed accurately. Their error analysis established precise conditions under which mixed-precision methods deliver both performance gains and reliable convergence, making this paper a central reference for modern mixed-precision numerical algorithms [11].

Chapter 3

Rounding in Floating Point Arithmetic

One of the core discussions in this thesis is the comparison between round-to-nearest (RTN) and stochastic rounding (SR). In this chapter, we compare them from a floating point arithmetic perspective and discuss each of their advantages and disadvantages. Moreover, we go through the Julia SR package that will be used, including how it works and its limitations.

3.1 Round-to-Nearest and Stochastic Rounding

The standard float point rounding method for IEEE floating point arithmetic is round-to-nearest (RTN), which is a deterministic approach that gives no variation [12]. With a fixed algorithm and problem, the result will be the same no matter how many times it is run.

As mentioned in the introduction, SR randomly rounds a number to one of its two nearest float point neighbors with probabilities proportional to their distances. In this case, the rounding is unbiased with no error in expectation. In Chapter 2, two highlighted papers discussed the error bound of SR compared to RTN, indicating

why it is a better rounding method in some circumstances, especially low precision arithmetic.

Based on the fundamental axiom of floating point arithmetic, for all $x, y \in \mathbb{R}$, there exist ϵ with $|\epsilon| \leq u$ where $u = \frac{1}{2}\epsilon_{machine}$ is the unit round off which is half the machine precision, such that

$$\text{fl}(x \ \text{op} \ y) = (x \ \text{op} \ y)(1+\epsilon), \quad |\epsilon| \leq u, \quad \text{op} \in \{+, -, \times, /\}. \tag{3.1}$$

For SR, we can replace u with $2u$ [3],

$$\text{fl}_{SR}(x \ \text{op} \ y) = (x \ \text{op} \ y)(1+\delta), \quad |\delta| \leq 2u, \quad \text{op} \in \{+, -, \times, /\}. \tag{3.2}$$

For SR, the rounding errors $\delta_1, \delta_2, \ldots$ are random variables of mean zero such that

$$\mathbb{E}[\delta_i \mid \delta_{i-1}, \ldots, \delta_1] = \mathbb{E}[\delta_i] = 0. \tag{3.3}$$

For a single floating-point operation under SR, the rounding error satisfies $|\delta_i| \leq 2u$. However, in a computation involving n operations, the total error depends on how these local errors accumulate. Under RTN, the errors may accumulate in a consistent direction, which leads to a worst-case bound proportional to nu. Under SR, the rounding errors satisfy $\mathbb{E}[\delta_i] = 0$, so their accumulation behaves more like a random walk than a deterministic sum.

In particular, the sum of n independent zero-mean errors has variance proportional to nu^2, written as $O(nu^2)$. The $\mathcal{O}(\cdot)$ (big-O) notation means that the quantity is proportional to its argument, up to a constant factor. Since the typical magnitude of a random variable is proportional to the square root of its variance, the accumulated error is therefore of order $\sqrt{n}u$ rather than nu. This intuition is formalized in the probabilistic backward error analysis, where the classical nu term is replaced by a

bound of order $\sqrt{n}u$ with high probability [2][3].

Another key element in SR is the random number generator (RNG). The rounding decision of which float point number to take in SR is made probabilistically based on their relative distances, which requires a RNG to determine. However, running RNG at each operation level is time-consuming as it requires an additional computation step than RTN. Theoretical speaking, parallel computation for RNG is possible in a GPU, but the numerical experiments in this thesis will only be operated in the CPU.

In sum, SR brings advantages to low precision calculation as it ensures that rounding errors are unbiased in expectation. This allows small numerical updates to be occasionally preserved rather than consistently discarded. As a result, SR can help reduce stagnation and improve the behavior of iterative algorithms in low precision settings. This is why SR will only be applied to part of the algorithm when using Float16 (half precision), rather than to the entire computation.

3.2 Stochastic Rounding Packages

In this thesis, we will be using the StochasticRounding.jl package from Julia to implement SR in the algorithms [13]. As stated in the introduction, all numerical experiments will be run on an NVIDIA H100 computing system at Emory. We will use only the CPU, as the package is not compatible with GPU or CUDA execution.

The package provides four SR formats: Float64sr, Float32sr, Float16sr, and BFloat16sr, which behave like standard floating-point types but apply SR. Arithmetic at each operation is performed in a higher precision and then stochastically rounded to the target precision (e.g., Float16sr uses Float32). In this thesis, only Float16sr is used in the numerical experiments.

The logic of the package is simple. For SR to be the rounding mechanism in a series of operations, the variables involved must be represented in an SR format. As

in RTN, the precision of the result follows the variable with the highest precision. RTN rounding has priority over SR rounding when mixed formats are used, meaning that if even one variable (e.g. a fixed integer) is not in SR format, the operation will not be performed with SR and the resulting variable will not be in SR format.

There are clear limitations of this package. First, it does not perform true SR but rather a semi-SR approach, where the arithmetic operations are carried out using RTN in a higher precision and then stochastically rounded to the target precision. However, this still simulates the behavior of SR and can be regarded as an approximation of true SR. Another limitation, as mentioned earlier, is that it cannot be executed on GPU. This introduces runtime constraints since the RNG must be invoked individually at each operation when SR is applied. Nevertheless, the numerical experiments in this thesis focus primarily on the relative error and the accuracy of the results. The computational efficiency of SR is therefore not examined in these simulations.

In addition to the Julia StochasticRounding package used in this thesis, SR has also been implemented in several other languages. For example, MATLAB has *Stochastic Rounding Toolbox* and *chop*. Furthermore, if SR is implemented directly in C++ with CUDA, it can be executed on GPUs, where the RNG can theoretically be performed in parallel, potentially reducing the runtime of SR-based algorithms significantly.

Chapter 4

Iterative Refinement

In this chapter, we discuss the IR algorithms developed and how it changes based on the type of problems. We also explain regularization, from standard methods like Tikhonov to simply modifying the number of iterations in GMRES. Variations of IR algorithms are all a derivation of Algorithm 1. The focus of the modification is still on the step that solves update $\boldsymbol{d}$ as it is the most computationally intensive part of the algorithm, and it will also be the only step where SR is applied for comparison with RTN.

As previously mentioned, the algorithms will be solving the linear system, equation (1.1), or the Tikhonov regularization in blocked form, equation (1.2). Within a variation of the algorithm concerning GMRES, which will be discussed in Chapter 4.1.2, we will also attempt to solve the least square equation (1.4).

4.1 IR Algorithms

Since this thesis focuses on the comparison between two rounding methods, Algorithm 1 is modified so that SR can be applied in the step that solves the update $\boldsymbol{d}$. All algorithms discussed later will have two such versions, where the SR version should be expected in a similar fashion.

Algorithm 2 Given a non-singular matrix $\boldsymbol{A} \in \mathbb{R}^{n\times n}$, $\boldsymbol{b} \in \mathbb{R}^n$, and an initial guess $\boldsymbol{x}_1$, this algorithm uses mixed-precision iterative refinement and stochastic rounding to solve $\boldsymbol{Ax} = \boldsymbol{b}$. Three precisions are used in the algorithm where $u_r \leq u \leq u_l$.

1 Define a new variable $\boldsymbol{A}_{sr}$ to be $\boldsymbol{A}$ in SR format and precision u_l.
2 for $i = 1 : i_{\max}$ or until relative residual reaches tolerance.
3 Compute $\boldsymbol{r}_i = \boldsymbol{b} - \boldsymbol{Ax}_i$ in precision u_r.
4 Let $\boldsymbol{r}_i$ be in SR format and precision u_l.
5 Solve $\boldsymbol{A}_{sr}\boldsymbol{d}_i = \boldsymbol{r}_i$ at precision u_l in stochastic rounding.
6 Update $\boldsymbol{x}_{i+1} = \boldsymbol{x}_i + \boldsymbol{d}_i$ in precision u.
7 end

In this algorithm, for the calculation of $\boldsymbol{d}$ to be performed in SR, both variables have to be in SR format. However, if we define $\boldsymbol{A}_{sr}$ in the loop, it will be different every time, adding uncertainty to the algorithm, thus it is defined outside the loop. When it comes to updating the solution, the addition doesn't have to be performed in SR, and since $\boldsymbol{x}_i$ is not in SR format, the operation will be performed in RTN.

4.1.1 Factorization

The first modification of Algorithm 1 is to use factorization. Higham and Mary defined an algorithm named *LU-IR* that uses LU factorization to simplify the process of solving the update $\boldsymbol{d}$ [11]. One difference we have is that we kept the initial guess of $\boldsymbol{x}_1$, instead of using the solution to $\boldsymbol{LU}\boldsymbol{x}_1 = \boldsymbol{b}$ in precision u_l.

Algorithm 3 Given a non-singular matrix $\boldsymbol{A} \in \mathbb{R}^{n\times n}$, $\boldsymbol{b} \in \mathbb{R}^n$, and an initial guess $\boldsymbol{x}_1$, this algorithm uses mixed-precision iterative refinement to solve $\boldsymbol{Ax} = \boldsymbol{b}$. Three precisions are used in the algorithm where $u_r \leq u \leq u_l$.

1 Compute $\boldsymbol{A} = \boldsymbol{LU}$ in precision u_l.
2 for $i = 1 : i_{\max}$ or until relative residual reaches tolerance
3 Compute $\boldsymbol{r}_i = \boldsymbol{b} - \boldsymbol{Ax}_i$ in precision u_r.
4 Solve $\boldsymbol{LU}\boldsymbol{d}_i = \boldsymbol{r}_i$ at precision u_l.
5 Update $\boldsymbol{x}_{i+1} = \boldsymbol{x}_i + \boldsymbol{d}_i$ in precision u.
6 end

For the SR version, all is similar just that the LU factorization will be in SR format. In Julia, there is a function $lu()$ that can store the LU in an implicit way. If the input has SR format, the implicit form will also operate in stochastic rounding when later solving the update $\boldsymbol{d}$.

This can be taken advantage of, extending the algorithm to solve $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ by using QR factorization, $qr()$. Everything else works the same.

Unfortunately, Julia doesn't have a built in function to store the implicit form of SVD factorization, but LU and QR can work efficiently with most types of matrices.

4.1.2 GMRES

Another approach is to use an iterative solver to compute the update $\boldsymbol{d}$. One of the most efficient ways to solve square linear systems is to use GMRES.

GMRES (Generalized Minimal Residual) is a Krylov subspace method that iteratively constructs an approximate solution by minimizing the residual over a growing Krylov subspace generated by the system matrix and the initial residual. At iteration k, the method seeks the solution in the space $\mathcal{K}_k(\boldsymbol{A}, \boldsymbol{r}_0)$ where $\boldsymbol{r}_0$ is the initial residual. The Arnoldi process within GMRES is used to generate an orthonormal basis of this Krylov subspace and to transform the original problem into a smaller upper Hessenberg least squares problem, which GMRES solves at each iteration to obtain the update [16].

In the algorithm, we define an additional function $GMRES(\boldsymbol{A}, \boldsymbol{r})$ that solves $\boldsymbol{Ad} = \boldsymbol{r}$ and returns the value $\boldsymbol{d}$ as the solution to the least square problem, which is also the update for the linear equation $\boldsymbol{x}$. With a similar structure as the ones before, the algorithm now has this form, given in Algorithm 4.

Again, for the SR version, the input of $\boldsymbol{A}$ in the GMRES function must be a pre-defined $\boldsymbol{A}_{sr}$ that has SR format and u_l precision, like step 1 in Algorithm 2.

At the same time, the GMRES function available in Julia as additional packages

Algorithm 4 Given a non-singular matrix $\boldsymbol{A} \in \mathbb{R}^{n\times n}$, $\boldsymbol{b} \in \mathbb{R}^n$, and an initial guess $\boldsymbol{x}_0$, this algorithm uses mixed-precision iterative refinement to solve $\boldsymbol{Ax} = \boldsymbol{b}$. Three precisions are used in the algorithm where $u_r \leq u \leq u_l$.

1 for $i = 1 : i_{\max}$ or until relative residual reaches tolerance
2 Compute $\boldsymbol{r}_i = \boldsymbol{b} - \boldsymbol{Ax}_i$ in precision u_r.
3 Compute $GMRES(\boldsymbol{A}, \boldsymbol{d}_i) = \boldsymbol{r}_i$ at precision u_l with a predefined function.
4 Update $\boldsymbol{x}_{i+1} = \boldsymbol{x}_i + \boldsymbol{d}_i$ in precision u.
5 end

are unable to work with SR format, thus it is written separately. When writing additional functions, one must make sure that the variable type is consistent with the input variables. For example, in GMRES the residual is stored in a vector, and the initialization of this vector must use the same numeric type as the input variables. If SR is used, the residual vector should also be initialized in the SR format so that subsequent operations preserve the intended rounding behavior and precision.

GMRES is typically applied to square linear systems of the form $\boldsymbol{Ax} = \boldsymbol{b}$. In the case of a rectangular system arising from a least squares problem, GMRES can instead be applied to an equivalent square system like equation (1.2) for $\lambda = \mathbf{0}$. This allows the method to be used in the least squares setting without modifying its core structure.

4.2 Regularization

Regularization is a technique used to stabilize the solution of ill-conditioned problems by controlling the influence of numerical errors and noise. It modifies the original problem to favor solutions that are more stable or well-behaved, often by limiting extreme or oscillatory components. As a result, regularization helps prevent error amplification and produces solutions that are more reliable in practical computations.

Using regularization is a key element in this thesis and the standard regularization

method used here is Tikhonov regularization. Shown in the introduction, the original linear system is equation (1.1), and equation (1.2) regularized block form. With an appropriate choice of λ, the condition number of the original problem can be significantly reduced, making the problem more numerically stable and easier to solve accurately.

One important point is that the resulting block matrix represents a new system that is not directly comparable to the original problem, where it not only doubled the size of the matrix from both directions, it offers a special structure to the problem. Therefore, direct comparisons of the result produced by the two formulations may not be as meaningful.

In Tikhonov regularization, the parameter λ controls the balance between fitting the data and stabilizing the solution. If λ is too small, the regularization effect becomes weak and the solution may overfit the noise in the problem, producing unstable or highly oscillatory results. On the other hand, if λ is too large, the regularization becomes overly strong and the solution may be oversmoothed, failing to capture important features of the true solution.

Apart from the choice of the λ parameter, regularization can also appear implicitly in iterative methods. In Krylov methods such as GMRES, the number of iterations acts as a form of regularization. Early iterations capture the dominant components of the solution, while later iterations may begin to amplify noise. As a result, running GMRES for too many iterations can lead to semi-convergence, where the residual continues to decrease but the true error begins to increase, effectively overfitting the noise. Stopping the iteration early can act as an iterative regularization strategy.

Chapter 5

Numerical Experiments

This chapter focuses on the numerical examples we ran using algorithms provided in Chapter 4.1. There are two main experimental problems considered, ill-conditioned inverse heat matrices and a large-scale Kronecker product PRblur problem. The first serves as a test case to examine the general effect of SR on extremely ill-posed problems. The second addresses a specific image deblurring problem formulated with a Kronecker product, testing how SR performs in a more realistic setting. Noise is added to the second problem to observe how different rounding methods are affected.

For the PRblur example, a single-precision algorithm will also be implemented, where algorithms discussed in Chapter 4.1 will be performed in purely Float64, Float32, or Float16. These serve as a baseline comparison to illustrate the advantages of mixed-precision computation.

All plots compare relative error between updated solution to the true solution,

$$\frac{\|\boldsymbol{x}_k - \boldsymbol{x}_{\text{true}}\|}{\|\boldsymbol{x}_{\text{true}}\|}, \quad k = 1, 2, \ldots, iter_{max}. \tag{5.1}$$

The first plotted iteration is actually iteration 0, which represents the relative error of the initial guess. It is included to standardize the graphs. Therefore, each graph shows one additional iteration beyond the number of iterations actually performed.

The programming language used is **Julia (version 1.12)**. Computations are performed on the CPU of an **NVIDIA H100** system connected to the Emory ADA server through VS Code. Julia provides native support for Float16 types, allowing closer control of half precision computations compared to MATLAB and Python environments, where operations are commonly promoted to higher precision before rounding back to half precision.

Other than the plot of relative error, the lowest relative error is also compared, including its value and the iteration at which it occurs. This helps to see whether more iterations are needed for better convergence or if the results diverge after some iterations. This is a useful examination when a certain level of noise is added.

For the implementation of Algorithm 4, the predefined GMRES function is run with 15 iterations. During the experiments, I observed divergence when the number of GMRES iterations exceeds 30, while the best results are typically obtained when the iteration count is between 15 and 20. This may be related to the implicit regularization discussed in Chapter 4.1.2.

Moreover, for all regularization parameters λ used, the value is chosen in the range 0.1 to 0.01 for the best performance. In most cases, $\lambda = 0.06$ is a sufficiently good choice, but for the PRblur problem with high noise, a larger λ can be used to better suppress the noise. In general, the convergence behavior remains similar across these choices, with only a small difference in the lowest relative error achieved.

All codes are displayed on the GitHub page named **SR_mixIR** and can be found here: (`https://github.com/WallGuan/SR_mixIR`)

5.1 Ill-conditioned inverse heat matrix

The construction of this problem uses a translation of the MATLAB code *heat.m* from the *regtools* file [10]. The function inputs the size and a parameter κ of the

inverse heat matrix, and outputs matrix $\boldsymbol{A}$, vector $\boldsymbol{b}$, and true solution $\boldsymbol{x}$. The lower the κ or the larger the matrix, the more ill-conditioned the problem is.

For this problem, both Algorithm 3 and Algorithm 4 will be tested and compared side by side. Since the problem is extremely ill-conditioned, regularization will be applied or it cannot be processed by the algorithm.

In this experiment, we used the default κ value of 1.0, and matrix size of 256. This results in an extremely ill-posed problem with $cond(\boldsymbol{A}) \approx 10^{18}$. The initial guesses will be the vector of ones. After regularization, the problem becomes

$$\boldsymbol{A}_{reg} = \begin{bmatrix} \boldsymbol{I} & \boldsymbol{A} \\ \boldsymbol{A}^T & -\lambda^2 \boldsymbol{I} \end{bmatrix} \in \mathbb{R}^{512\times 512}, \quad \boldsymbol{b}_{reg} = \begin{bmatrix} \boldsymbol{b} \\ \boldsymbol{0} \end{bmatrix}, \quad \boldsymbol{x}_{reg} = \begin{bmatrix} \boldsymbol{0} \\ \boldsymbol{x} \end{bmatrix}.$$

The plot below uses Algorithm 3 and Algorithm 4. The algorithm will run once for RTN and run three times for SR as each run will produce slightly different results.

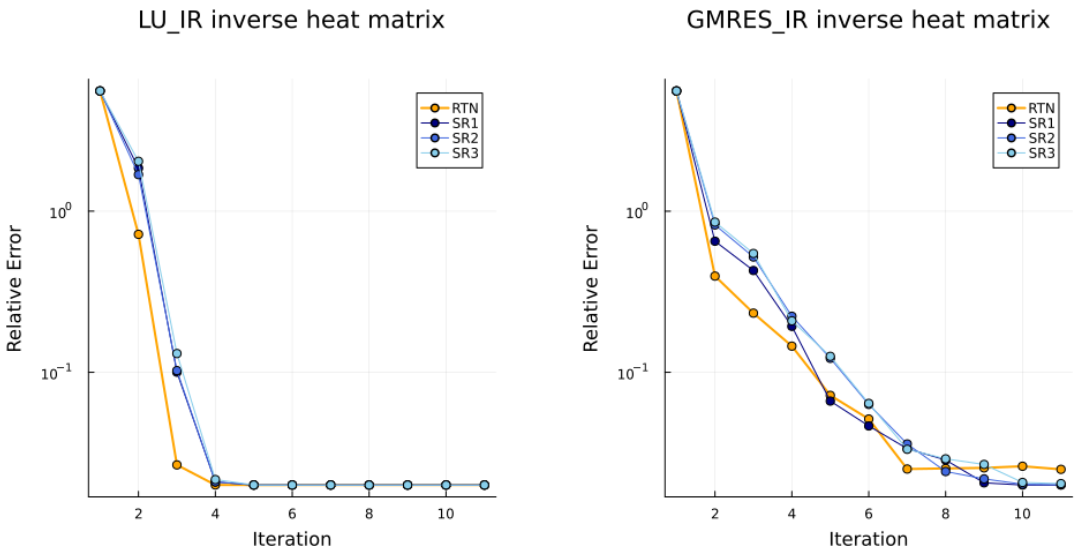


Figure 5.1: Both methods converge, and the SR runs remain relatively stable. Compared with GMRES, the LU-based algorithm converges faster.

The plot on the left is the result produced by Algorithm 3. We first observed that both RTN and SR converge very quickly, reaching their limits at about the 3rd to

4th iteration. The lowest relative error is approximately 0.02. There is barely any difference between the performance of RTN and SR, with RTN showing slightly faster convergence at the very beginning, though both reach the lowest value at roughly the same iteration. One interpretation is that SR behaves very stable, as all three SR runs almost completely overlap with each other.

Moving to the graph on the right, which corresponds to Algorithm 4, more variations can be observed. In general, the performance remains similar, with RTN producing a slightly higher relative error at 0.025 compared to 0.02 reached by the SR runs. All methods reach their lowest relative error around iteration 10, suggesting that increasing the number of iterations might further reduce the error. One interesting observation is that after iteration 7, SR appears to slightly outperform RTN, although the difference is small and does not allow for a strong conclusion. What can be noted is that SR continues to behave relatively stably across different runs.

In summary, for a relatively small matrix such as this one (after regularization, the matrix size is 512×512), the LU-based IR converges much faster because it solves the system directly through factorization rather than using an iterative solver. Overall, there is essentially no difference in performance between RTN and SR, and SR remains stable across repeated runs.

5.2 PRblur Kronecker product

This example is a specific image deblurring problem from the MATLAB *IRtools* package. Similar to the approach in Chapter 5.1, the main comparison will be between relative error. This experiment is separated into two parts, one is noise free and one with noise added. The deblurred image will also be shown for visualization.

A special characteristic of this problem is that it is constructed using a Kronecker product, as the original problem is very large. Working with Kronecker products

requires defining additional objects to compute them implicitly, which helps to reduce computational costs.

In the unregularized case, an LU factorization can be used when the operator has an exact Kronecker product structure. I did not implement the Kronecker-based LU approach because the regularized case becomes much more complicated and harder to implement, making comparison more difficult. Therefore, only Algorithm 4 will be used here. Single precision performance will also be tested as baseline.

5.2.1 Kronecker product definition

Before moving on to the results, explanation of the Kronecker product object defined in Julia needs to be emphasized. The codes are another translation of the MATLAB Kronecker product functions in IRtools.

The Kronecker product is an operation between two matrices that constructs a larger block matrix from their entries. Specifically, given matrices

$$\boldsymbol{A} \in \mathbb{R}^{m\times n}, \qquad \boldsymbol{B} \in \mathbb{R}^{p\times q}, \tag{5.2}$$

their Kronecker product is denoted by $\boldsymbol{A} \otimes \boldsymbol{B}$ and is defined as

$$\boldsymbol{A} \otimes \boldsymbol{B} = \begin{bmatrix} \boldsymbol{a}_{11}\boldsymbol{B} & \boldsymbol{a}_{12}\boldsymbol{B} & \cdots & \boldsymbol{a}_{1n}\boldsymbol{B} \\ \boldsymbol{a}_{21}\boldsymbol{B} & \boldsymbol{a}_{22}\boldsymbol{B} & \cdots & \boldsymbol{a}_{2n}\boldsymbol{B} \\ \vdots & \vdots & \ddots & \vdots \\ \boldsymbol{a}_{m1}\boldsymbol{B} & \boldsymbol{a}_{m2}\boldsymbol{B} & \cdots & \boldsymbol{a}_{mn}\boldsymbol{B} \end{bmatrix} \in \mathbb{R}^{mp\times nq}. \tag{5.3}$$

This construction is useful because it allows large structured matrices to be built from smaller factors. In numerical linear algebra, Kronecker products are often used to represent multi-dimensional problems compactly and to reduce storage or computational cost when the structure can be exploited implicitly.

The Kronecker product has several useful properties, such as mixed product, transpose, and compatibility with vectorization, which can be exploited to perform computations implicitly without explicitly forming the large Kronecker product matrix. These are used to define the *KronMatrix* object in Julia, allowing computation to work around the Kronecker factors. An additional definition added is precision, allowing compatibility between the object definition and the SR format.

Another note on the definition is regularization. For Kronecker product, it is hard to construct the block matrix then factor it down. Thus, the explicit form of the regularized Kronecker product is not formed, but a function *AugMatVecMult* is used. This function outputs the value of $\boldsymbol{A}_{kron}\boldsymbol{b}$ to bypass the formation of the block matrix. This function is not part of the object definition.

5.2.2 Noise free condition

This section discusses the results with no additional noise added to the problem, or noise free. There will be two plots shown here, one is the relative error convergence by single precision and one by mixed-precision.

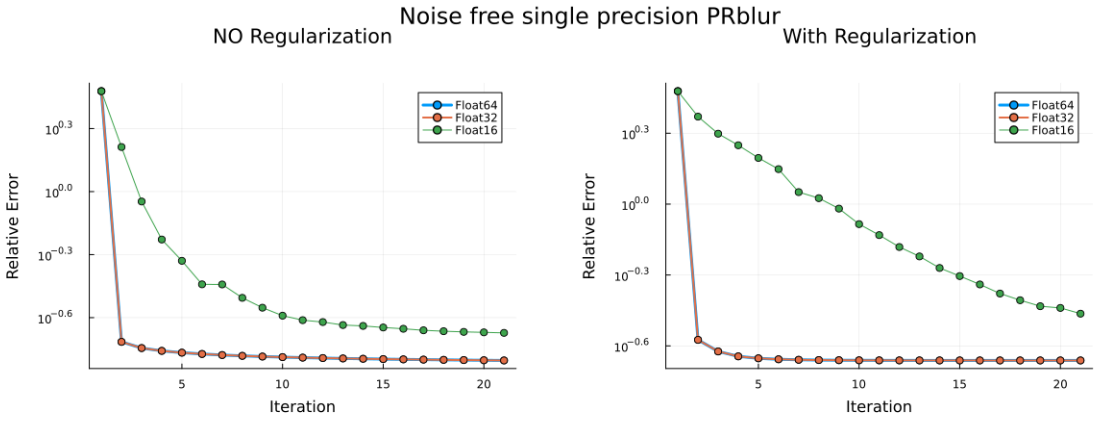


Figure 5.2: In the noise-free case, regularization slightly complicates the problem and requires more iterations for Float16 to converge, but it does not affect the behavior of the high-precision computation.

Without any additional noise added, regularization doesn't make a big difference and complicates the problem, requiring more iterations to converge to the lowest relative error. From both graphs, Float64 and Float32 perform identically. Float16 converges slower with an accuracy limitation.

On average, lowest relative error for no regularization is around 0.157, corresponding to 0.218 for the regularized case. Respectively, Float16 has 0.213 and 0.344.

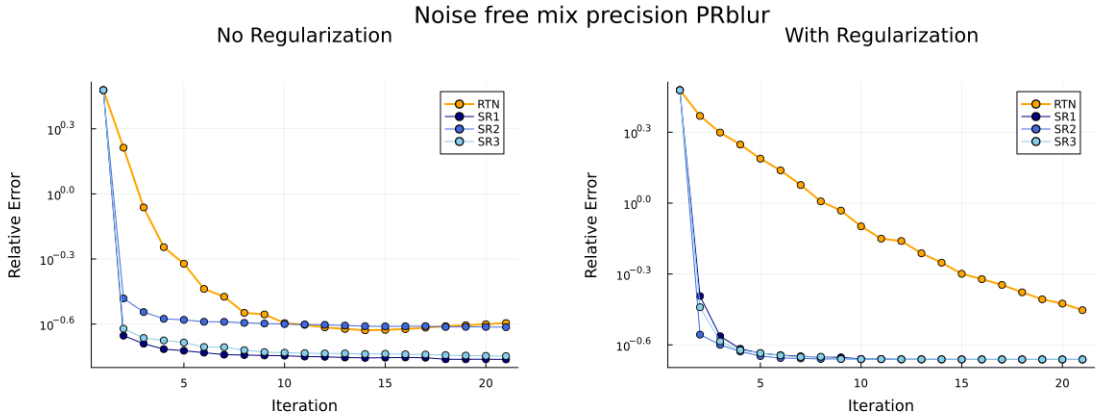


Figure 5.3: SR cases performance are more stable after regularization, and RTN case converge slower after regularization.

Similarly for the mixed-precision case, without added noise, regularization doesn't affect the output too much but only complicates the problem. Also, SR performs better than RTN in both cases. Even though the value didn't decrease much as iterations increase, SR produces a much lower relative error in comparison.

Regularization increases the numerical value of RTN case from 0.235 to 0.352. The lowest reached by SR is 0.173 and 0.218 respectively.

It is also noticeable that SR behaves similarly to that of pure Float64/Float32, and RTN behaves similarly to that of pure Float16. This indicates the advantage of SR in mixed-precision algorthms, but also poses questions based on how the SR package functions. More will be discussed in Chapter 6.

In sum, without additional noise added, regularization is unnecessary. Through comparison, the performance of single precision on Float64 is similar to that of mixed-precision SR, indicating the advantage of using SR in iterative algorithms. The SR implementation on CPU incurs a significantly higher runtime ($80 \sim 160$s per run versus ≈ 5s for the baseline), but this runtime analysis is not meaningful as the overhead mainly comes from the CPU implementation and would likely not persist in a native GPU implementation.

5.2.3 Noise added conditions

Here, we introduce noise to the problem and see how performance changes. Gaussian noise is used here with three levels: small (10^{-3}), medium (10^{-2}), and large (10^{-1}).

With noise added, the importance of regularization becomes evident, highlighting that the decision to use it depends on the type of problem. Even a small amount of noise causes the un-regularized problem to diverge, while a large amount of noise can also lead to slight divergence in the regularized case, as shown in Figure 5.6.

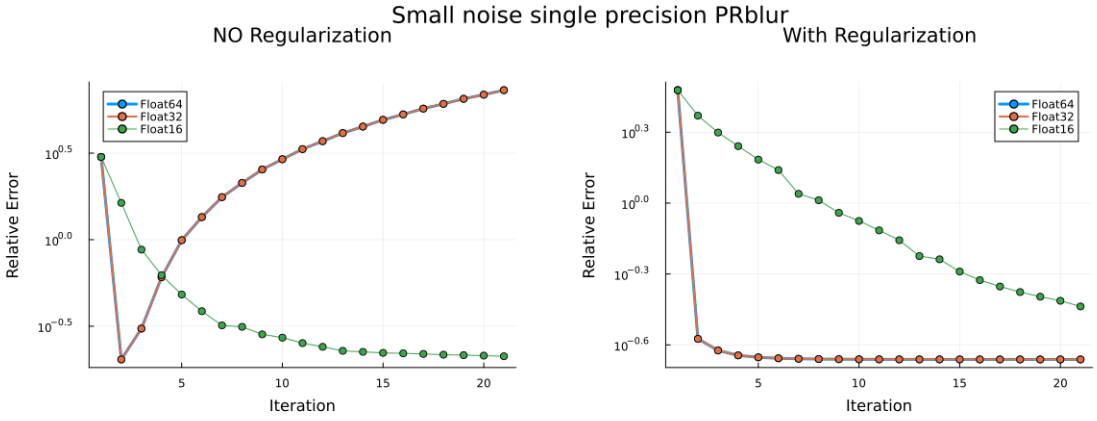


Figure 5.4: Without regularization, the high-precision computation diverges after first iteration, while interestingly the Float16 computation converges. With regularization, high-precision computation converges faster.

One interesting observation is that for the no regularization case, Float16 performs much better. Instead of diverging, it is slowly converging and reaches a decently low

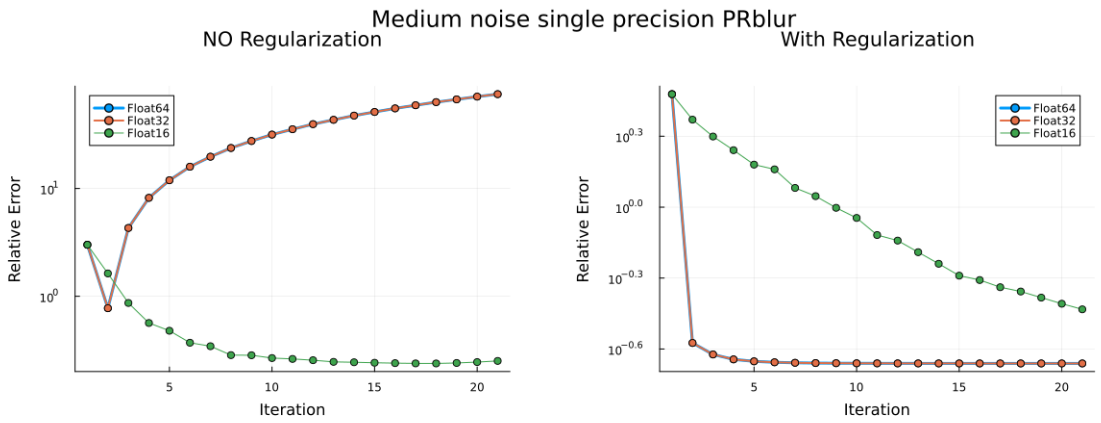


Figure 5.5: Without regularization, the high-precision computation diverges after the first iteration, while interestingly the Float16 computation converges. With regularization, the high-precision computation converges faster.

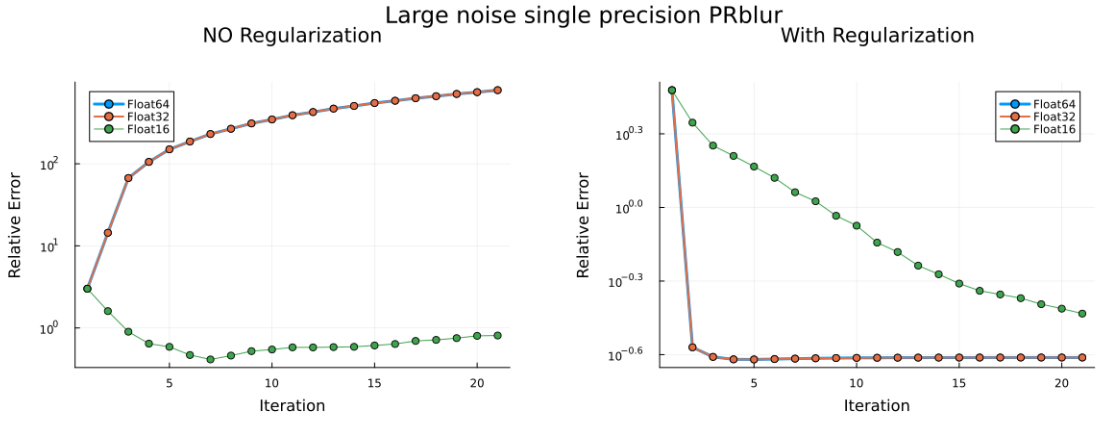


Figure 5.6: Without regularization, the high-precision computation diverges immediately, and the Float16 computation shows slight divergence after iteration 7. With regularization, convergence is restored, indicating that the chosen regularization parameter effectively suppresses noise.

relative error. A possible explanation could be that low precision computation introduces larger rounding errors that may act as a form of implicit regularization, suppressing the amplification of noise. This phenomenon is worth investigating further to better understand its mechanism and whether it can be systematically utilized in solving un-regularized ill-posed problems.

For the small-noise case on Figure 5.4, the lowest relative error reached by the

regularized Float64 computation is approximately 0.218. The value for the medium-noise case is 0.218 and 0.240 for the large noise case.

In these setting, the regularized Float64 result can serve as a baseline for evaluating the performance of the mixed-precision SR approach.

Moving on to the mixed-precision case, the non-regularized output is not suitable for discussion. The SR runs appear quite inconsistent and exhibit noticeable fluctuations. Although the RTN case shows some degree of convergence, the minimum relative errors it achieves are generally higher than those obtained by the SR runs, except when the input noise is large on Figure 5.9.

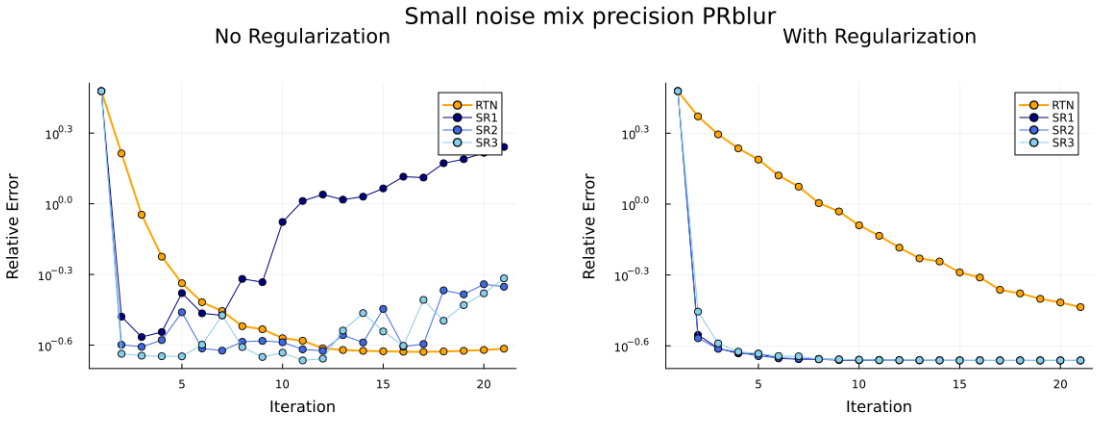


Figure 5.7: Without regularization, the SR cases fluctuate much more and show a diverging trend. With regularization, the SR cases are more stable and reach a lower relative error than the RTN case.

For the regularized case, the graph closely resembles the behavior observed in the single precision experiment. In particular, the RTN curve behaves similarly to the pure Float16 computation, whereas the SR runs resemble the behavior seen in the Float64 and Float32 cases.

The observation occured in the noise free case in Chapter 5.2.2. It suggests that in mixed-precision computation, even when only the update step is performed in low precision, the overall behavior of the solution can resemble that of a pure low-precision computation. This highlights the importance of that step within the IR algorithm.

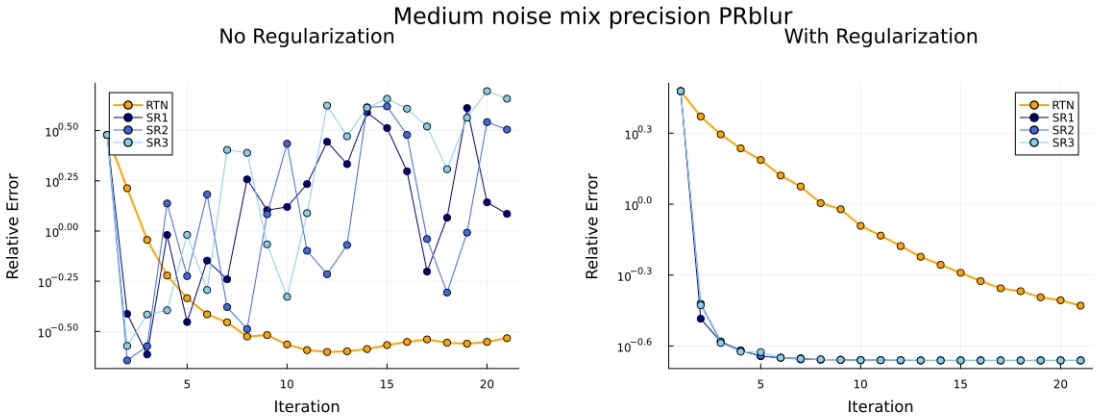


Figure 5.8: Without regularization, the SR cases fluctuate much more and show a clear diverging trend. With regularization, the SR cases become more stable and reach a lower relative error than the RTN case.

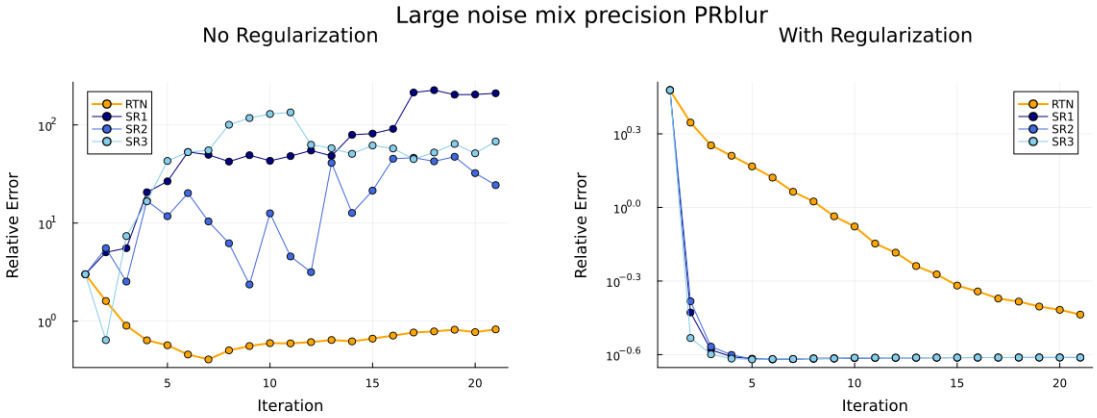


Figure 5.9: Without regularization, the SR cases fluctuate strongly and exhibit a clear diverging trend. With regularization, the SR cases are significantly more stable and attain a lower relative error than the RTN case.

Looking at the numbers, small noise SR has a lowest relative error of 0.218 compare to 0.367 from RTN. For medium noise, those values are 0.218 and 0.372, and 0.240 and 0.367 for large noise respectively.

These values indicate that SR performs similar to pure high precision computation, outperforming RTN mixed-precision is almost every single case in this example.

5.2.4 Image display

This section presents the restored images produced by the algorithms, visualizing how different methods perform. The *MAT* package is used to output *.mat* files of the results, and the images are displayed in MATLAB.

The visualizations includes noise free and large noise outcome for mixed-precision RTN and SR.

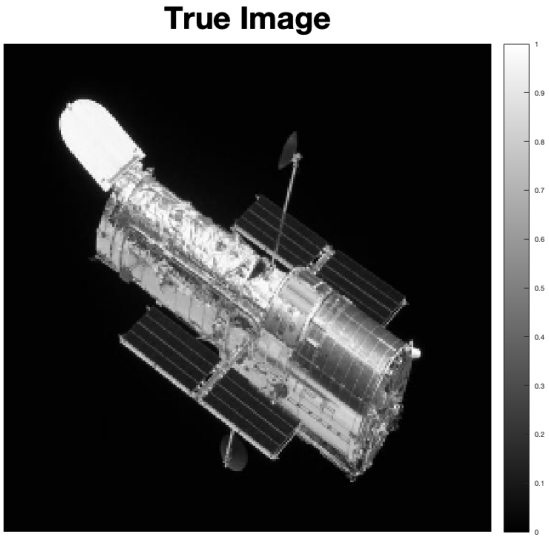


Figure 5.10: This is the true image of the PRblur problem.

Based on the output, the closest resemblance to the true image is achieved by the noise free SR case, while the worst performance is observed in the large noise RTN case. The differences between the other two cases are not very significant, but the large noise SR case contains more visible noise spots, which is expected given the high noise level in the input.

In general, similar to what the relative error plots in Chapter 5.2.2 and Chapter 5.2.3 suggested, SR out perform RTN in mixed-precision algorithms and produces better outcomes.

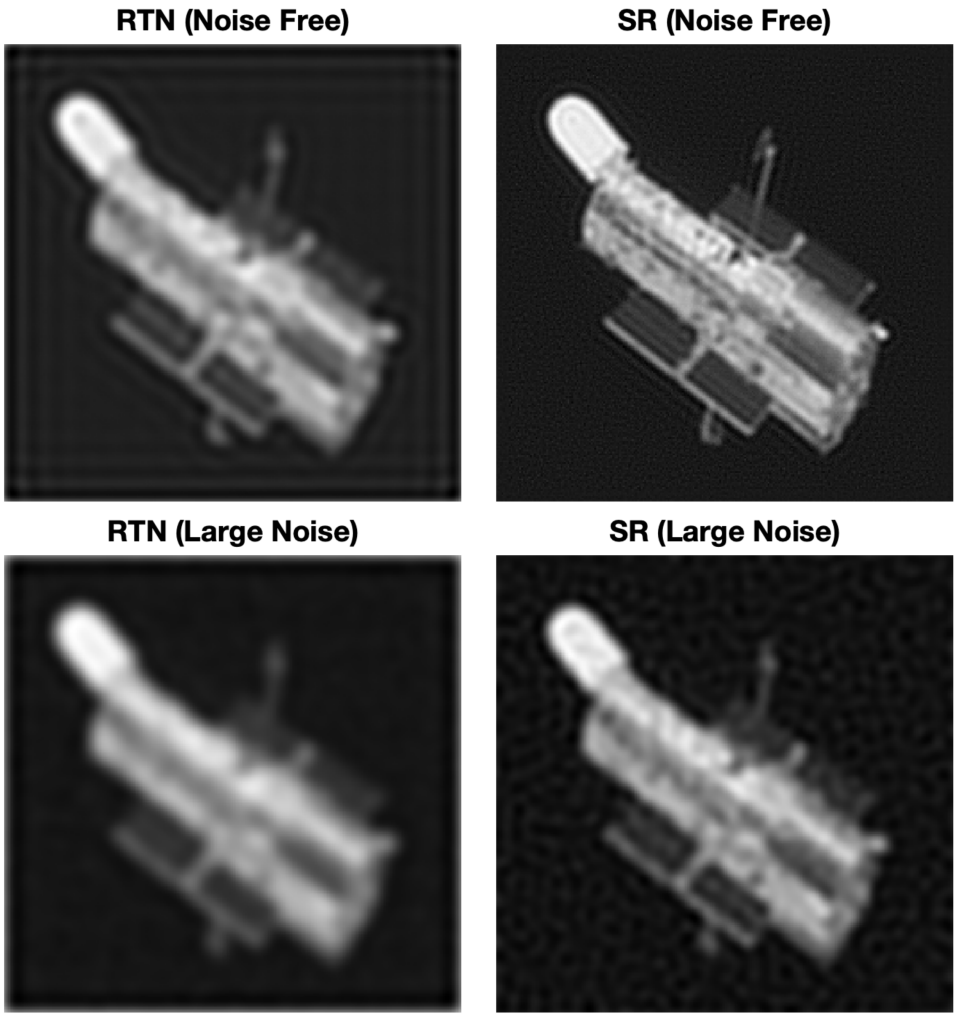


Figure 5.11: This is the restoration for the noise free and large noise cases.

Chapter 6

Concluding Remarks

In this thesis, we investigated the use of stochastic rounding in mixed-precision iterative refinement algorithms. We first discussed how linear equations and least squares problems can be connected, including Tikhonov regulation block form. Then we moved onto literature reviews, discussing past findings in this field that build up to the investigation in this thesis. Moving on, we extensively discussed rounding in modern computation, including how IEEE float point arithmetic works and the difference between round-to-nearest (RTN) and stochastic rounding (SR). After that we went through a series of iterative refinement algorithms proposed by others and the alternative form that we used here. Finally, two numerical experiments were conducted: a generic ill-posed inverse heat problem and a PRblur Kronecker product image deblurring problem. The latter was tested under both noise and no-noise settings, while the former focused on a generic ill-posed case.

For the inverse heat experiment, the results show that SR and RTN behave similarly. In this setting, the advantage of SR is not significant, and both rounding modes achieve comparable levels of accuracy. This suggests that for certain ill-posed problems, the choice of rounding mode may not strongly influence the performance of the IR algorithm.

In contrast, the PRblur image deblurring experiment shows clearer differences between the two rounding strategies. In particular, under small and medium noise settings, SR demonstrates advantages over RTN by reducing stagnation and producing smaller relative errors in several runs. At the same time, SR runs often resemble the behavior of higher-precision computation, while RTN behaves more similarly to pure low-precision arithmetic. This observation suggests that the rounding strategy used in the update step of IR can significantly influence the overall behavior of the algorithm.

Throughout this work we explored several IR strategies. Based on findings from previous literature, algorithms using LU and QR factorizations were considered, and GMRES was used within the IR framework to efficiently solve large-scale systems. The use of GMRES places an iterative method within the iterative refinement framework. This enables the method to scale to larger problems while maintaining stable convergence properties.

Regularization also plays an important role in the experiments. Explicit regularization, such as the block matrix form of Tikhonov regularization used in the numerical examples, helps reduce the condition number of extremely ill-posed systems and mitigates the effect of noise. In addition, implicit forms of regularization were observed. For example, adjusting the number of GMRES iterations can influence stability, and in some high-noise cases, pure low-precision computation appears to converge rather than diverge. This behavior may be interpreted as a form of implicit regularization that limits noise amplification, and further investigation can be done to better understand this effect.

There are also limitations in the current implementation. As discussed in Chapter 3.2, the StochasticRounding.jl package used in Julia performs pseudo-SR, where arithmetic operations are first carried out in higher precision then stochastically rounded to target precision. As a result, the computation of those steps is effectively per-

formed in a higher precision. Although the numerical experiments show that SR behaves similarly to higher-precision computation, further investigation is needed to determine whether true SR would produce the same behavior.

There are two main future directions that follow from this thesis. First, a C++ implementation with CUDA support could provide native GPU execution of SR, allowing true SR with parallel RNG to better study convergence behavior and runtime performance. Second, the convergence of pure low-precision computation in noisy problems suggests that low precision may act as a form of implicit regularization by suppressing unstable error growth, which deserves further investigation.

Overall, the experiments suggest that SR can be beneficial in mixed-precision IR, particularly in problems involving noise such as the PRblur image deblurring example. However, its advantage may depend on the structure of the problem and the numerical properties of the system being solved. Further study of SR in large-scale problems and under true low-precision arithmetic would help provide a deeper understanding of its role in mixed-precision numerical algorithms.

Bibliography

[1] Patrick Amestoy, Olivier Boiteau, Alfredo Buttari, Matthieu Gerest, Fabienne Jézéquel, Jean-Yves L'Excellent, and Theo Mary. Mixed precision low-rank approximations and their application to block low-rank LU factorization. *IMA Journal of Numerical Analysis*, 43(4):2198–2227, 08 2022.

[2] Michael P Connolly, Nicholas J Higham, and Théo Mary. Stochastic Rounding and its Probabilistic Backward Error Analysis. *SIAM Journal on Scientific Computing*, 43(1):A566–A585, February 2021.

[3] Matteo Croci, Massimiliano Fasi, Nicholas J. Higham, Theo Mary, and Mantas Mikaitis. Stochastic rounding: implementation, error analysis and applications. *Royal Society Open Science*, 9(3):211631, 03 2022.

[4] Matteo Croci and Michael B Giles. Effects of round-to-nearest and stochastic rounding in the numerical solution of the heat equation in low precision. *IMA Journal of Numerical Analysis*, 43(3):1358–1390, April 2022.

[5] James Demmel and Yozo Hida. Accurate and efficient floating point summation. *SIAM Journal on Scientific Computing*, 25(4):1214–1248, 2004.

[6] Lars Eldén. Algorithms for the regularization of ill-conditioned least squares problems. *BIT Numerical Mathematics*, 17(2):134–145, 1977.

[7] S. Gazzola, P. C. Hansen, and J. G. Nagy. IR Tools: a MATLAB package of

iterative regularization methods and large-scale test problems. *Numer Algor*, 81:773–811, 2019.

[8] David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Comput. Surv.*, 23(1):5–48, March 1991.

[9] Per Christian Hansen. Regularization tools: A matlab package for analysis and solution of discrete ill-posed problems. *Numerical Algorithms*, 6(1):1–35, 1994.

[10] Per Christian Hansen. Regularization tools version 4.0 for matlab 7.3. *Numerical Algorithms*, 46(2):189–194, 2007.

[11] Nicholas J. Higham and Theo Mary. Mixed precision algorithms in numerical linear algebra. *Acta Numerica*, page 347–414, 2022.

[12] IEEE. IEEE Standard for Floating-Point Arithmetic, July 2019.

[13] Milan Klöwer, Peter V. Coveney, E. Adam Paxton, and Tim N. Palmer. Periodic orbits in chaotic systems simulated at low precision. *Scientific Reports*, 13(1):11410, 2023.

[14] Cleve B. Moler. Iterative refinement in floating point. *J. ACM*, 14(2):316–321, 1967.

[15] James Nagy, Michael Ng, and Lisa Perrone. Kronecker product approximations forimage restoration with reflexive boundary conditions. *SIAM Journal on Matrix Analysis and Applications*, 25:829–841, 07 2006.

[16] Youcef Saad and Martin H. Schultz. GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 7(3):856–869, 1986.